

ECS 170
Artificial Intelligence
Spring 2026
Course Project: Stage 3 Report

Team Information

Enter Your Team Name Here (delete the extra rows if your team has less than 5 students)		
Student 1: Zoltan	Student 1: 924065079	Student 1: zlcabute@ucdavis.edu
Student 2: Roman Vasilyev	Student 2: 920502918	Student 2: rvasilyev@ucdavis.edu

Section 1: Task Description

The main task of this stage is to train a Convolutional Neural Network (CNN) model to classify image data for object recognition. Three distinct datasets were used: MNIST (handwritten digits), ORL (human faces), and CIFAR-10 (colored objects). The CNN model was built from scratch using PyTorch's nn.Module framework, following the standard CNN architecture of stacked convolutional blocks followed by fully connected classification layers.

The goal for each dataset was to train the model on the provided training split, then evaluate its performance on a held-out test split using accuracy, precision, recall, and F1 score as metrics.

Section 2: Model Description

The CNN architecture consists of two convolutional blocks followed by a fully connected classifier. The design follows the standard pattern of: convolution, activation, convolution, activation, then max pooling, repeated twice to extract increasingly abstract features before classification.

Architecture summary:

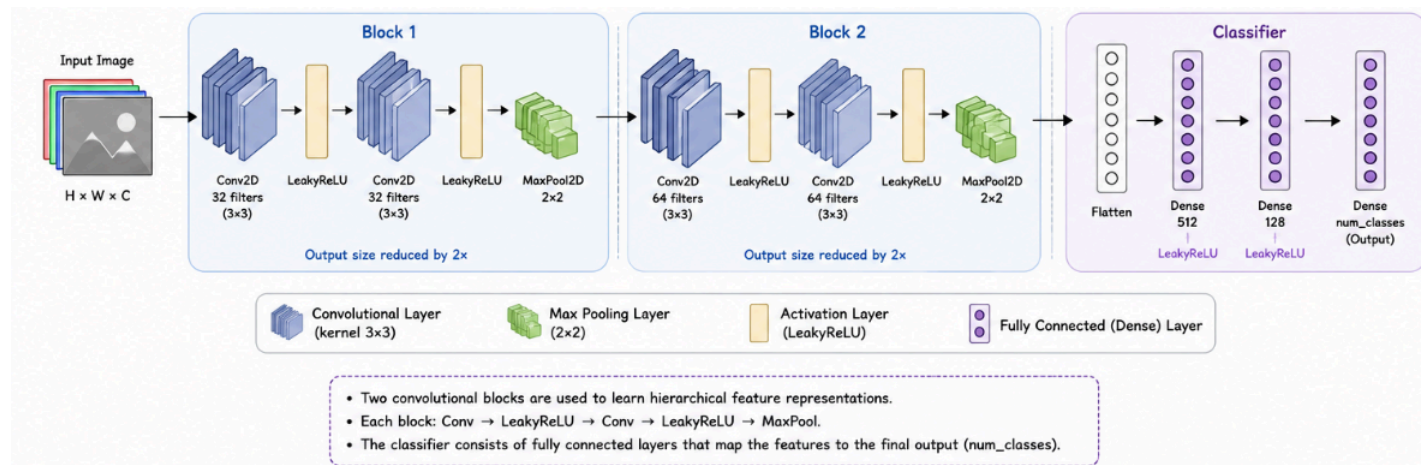
Block 1: Conv2d(in_channels, 32, kernel=3) → LeakyReLU → Conv2d(32, 32, kernel=3) → LeakyReLU → MaxPool2d(2x2)

Block 2: Conv2d(32, 64, kernel=3) → LeakyReLU → Conv2d(64, 64, kernel=3) → LeakyReLU → MaxPool2d(2x2)

Classifier: Flatten → LazyLinear(512) → LeakyReLU → Linear(512, 128) → LeakyReLU → Linear(128, num_classes)

The convolutional layers use learnable filters that slide across the image to detect local spatial patterns such as edges and textures. LeakyReLU activations introduce non-linearity while avoiding the dying ReLU problem by allowing a small negative gradient. Max pooling reduces spatial dimensions by keeping only the maximum value in each 2x2 region, making the model more robust to small translations. The flattening step converts the 3D feature maps into a 1D vector for the fully connected layers, which learn to map the extracted features to class probabilities.

(Please draw a plot about the model architecture and provide necessary description of the model used in your project)



Section 3: Experiment Settings

3.1 Dataset Description

There is no need to partition them, as they are already partitioned for us. We would be testing our models by first training them on the training set within each data set and then use the testing set to validate the accuracy of our model.

MNIST - a dataset of 70,000 handwritten digit images from 0 -9. Each image is 28 x 28 pixels in grayscale.

ORL - a dataset of human face photos from 40 different people, 10 photos per person. Each image is 112 x 92 pixels.

CIFAR-10 - a dataset of 60,000 small color photos of 10 different object categories of airplanes, cars, birds, cats, dogs, etc. Each image is 32 x 32 pixels in color

3.2 Detailed Experimental Setups

The same CNN architecture was used for all three datasets, with two parameters adjusted per dataset: in_channels (1 for grayscale MNIST/ORL, 3 for color CIFAR-10) and num_classes (10 for

MNIST/CIFAR-10, 40 for ORL). ORL labels were shifted from 1-40 to 0-39 to match PyTorch's 0-indexed class convention. I switched to 30 epochs because the original 100 was taking a long time.

Hyperparameter	MNIST	ORL	CIFAR-10
Optimizer	Adam	Adam	Adam
Learning Rate	0.001	0.001	0.001
Loss Function	CrossEntropyLoss	CrossEntropyLoss	CrossEntropyLoss
Epochs	30	30	30
Batch Size	64	64	64
in_channels	1	1	3
num_classes	10	40	10
Activation	LeakyReLU	LeakyReLU	LeakyReLU

3.3 Evaluation Metrics

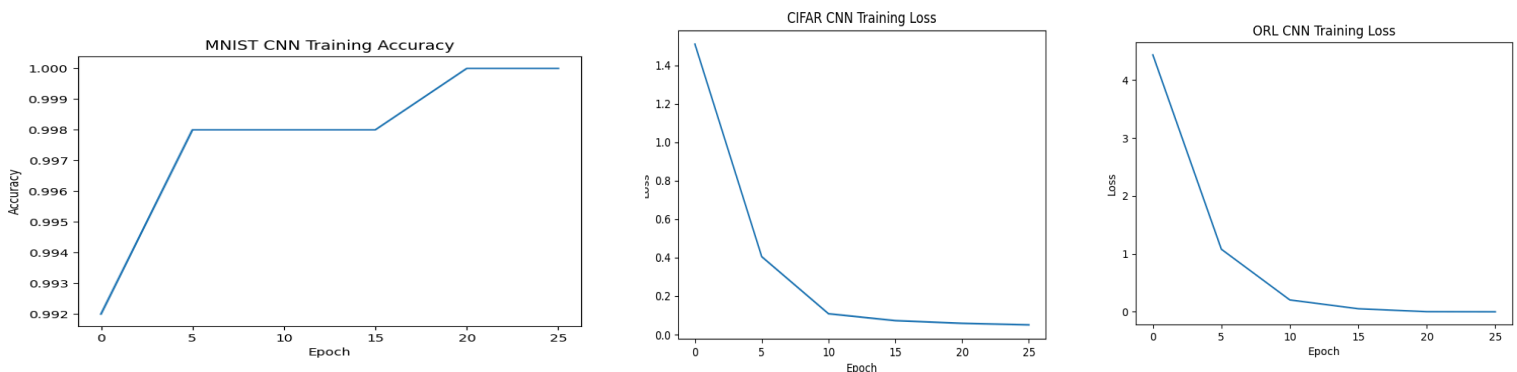
Four metrics were used to evaluate model performance. Accuracy measures the overall proportion of correctly classified samples. Precision measures how many of the predicted positives were actually positive. Recall measures how many actual positives were correctly identified. F1 Score is the harmonic mean of precision and recall, balancing both. All metrics used the weighted average across classes to account for class imbalance.

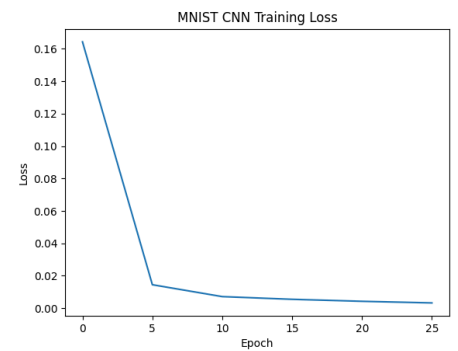
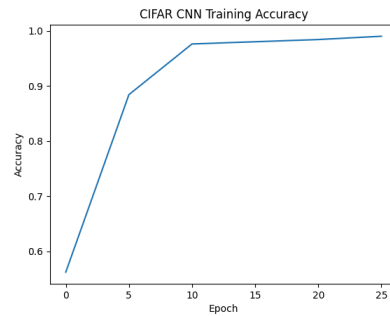
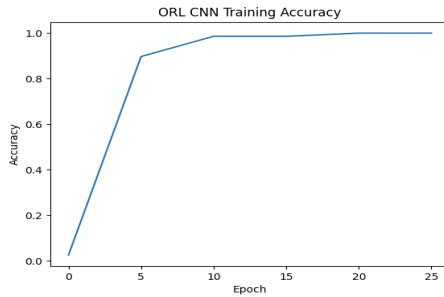
3.4 Source Code

<https://github.com/RomanV123/ECS-170-Machine-Learning-/tree/stage-3>

3.5 Training Convergence Plot

The plots below show the training loss and accuracy over 30 epochs for each dataset. All three models converge successfully with loss decreasing and accuracy increasing over time.





3.6 Model Performance

Dataset	Accuracy	F1 Score	Precision	Recall
MNIST	99.27%	0.9927	0.9927	0.9927
ORL	100%	1.0000	1.0000	1.0000
CIFAR-10	74.22%	0.7423	0.7449	0.7422

MNIST achieved 99.27% test accuracy, demonstrating that the CNN is well-suited to digit recognition. ORL achieved 100% accuracy, which is expected given the very small dataset size (360 training images) the model can effectively memorize the distinct features of each of the 40 subjects. CIFAR-10 achieved 74.22% test accuracy. Notably, training accuracy reached 99% while test accuracy was only 74.22%, indicating overfitting, the model memorized training patterns but did not generalize as well to unseen color images.

3.7 Ablation Studies

Configuration	LR	Kernel	Epochs	Test Accuracy
Baseline (no dropout, no BN)	1e-3	3	30	74.22%
+ Dropout (p=0.5)	1e-3	3	30	79.21%
+ Dropout, larger kernel	1e-4	5	30	71.67%
+ Dropout, smaller LR	1e-4	3	30	69.81%
+ Dropout, larger kernel, 100 epoch max	1e-4	5	100	77.66%
+ Dropout + BatchNorm, 100 epoch max	1e-4	3	100	82.51%

The ablation study focused exclusively on CIFAR-10 because it had the lowest baseline performance at 74.22% and showed a clear overfitting problem, training accuracy reached 99% while test accuracy was only 74.22%, a 25 point gap. MNIST and ORL were already performing at 99%+ so there was little room for meaningful improvement there. The goal was to systematically test different configurations to understand what was causing the poor generalization and how to fix it.

Step 1 — Baseline (74.22%) The starting point was the standard CNN with two conv blocks, LeakyReLU activations, Adam optimizer at $lr=1e-3$, kernel size 3, and 30 epochs. The high gap between training and test accuracy immediately indicated overfitting as the core problem.

Step 2 — Adding Dropout (79.21%) Dropout was added first because it directly targets overfitting. By randomly disabling 50% of neurons during each training pass, the model is forced to learn more robust and distributed features rather than memorizing specific patterns. This improved test accuracy by 5 percentage points and visibly closed the gap between training and test accuracy, confirming overfitting was the main issue.

Step 3 — Changing Kernel Size to 5 with $lr=1e-4$ (71.67%) Two changes were made together here — kernel size increased to 5 and learning rate reduced to $1e-4$. Both hurt performance. A larger kernel sees more of the image at once which can capture broader patterns, but the smaller learning rate meant the model updated its weights too slowly to benefit from it within 30 epochs. The model was still learning when training stopped.

Step 4 — Kernel size 3, $lr=1e-4$ only (69.81%) To isolate the effect of learning rate, kernel size was set back to 3 while keeping $lr=1e-4$. This confirmed that the learning rate reduction alone was the main culprit, 30 epochs simply was not enough time for the model to converge at this slower pace. Training accuracy only reached around 61%, showing the model had not finished learning.

Step 5 — Kernel size 5, $lr=1e-4$, 100 epochs (77.66%) Since the slow learning rate needed more time, epochs were increased to 100. This gave the model enough time to converge and performance recovered to 77.66%. Still below the dropout baseline of 79.21%, but it showed that larger kernels combined with slower learning and more training time can be a valid approach.

Step 6 — Dropout + Batch Normalization, $lr=1e-4$, 100 epochs (82.51%) Batch normalization was added on top of dropout, with kernel size back to 3. Batch normalization normalizes the output of each conv layer before the activation function, which stabilizes training and reduces sensitivity to the learning rate. Combined with dropout and 100 epochs this produced the best result of 82.51%, an 8.29 percentage point improvement over the baseline. The two techniques worked together: dropout prevented overfitting, and batch normalization helped the model train more stably and converge to a better solution.

The key lesson is that CIFAR-10's complexity requires regularization techniques. A plain CNN without dropout or batch normalization will memorize training data but fail to generalize. The best configuration combined dropout, batch normalization, a lower learning rate, and more epochs, each addressing a different aspect of the training problem.

